

Introduction

In the previous unit, we learned Arrays and performed some basic operations such as accessing and modifying them.

In this unit, we'll learn to perform some more operations like concatenating two arrays and finding subarrays. We will also learn about Multi-dimensional arrays.

1. Array Concatenation

Joining two arrays is called Array Concatenation. In Java, the

`System` class contains a method named `arraycopy()` to concatenate two arrays.

Syntax

JAVA

```
1 System.arraycopy(array1, array1StartPosition, array
```

Here,

- `array1` : is the name of the source array.
- `array1StartPosition` : indicates from which position in the source array the elements need to be copied.
- `array2` : is the name of the destination array
- `array2StartPosition` : indicates to which position in the destination array the elements need to be added to.
- `noOfElements` : is the number of array elements to be copied.

Code

JAVA

```
1 import java.util.Arrays;
2
3 class Main {
4     public static void main(String[] args) {
5         int[] arr1 = {12, 4, 5, 2, 5};
6         int[] arr2 = {6, 10, 11, 6};
7
8         int arr1Len = arr1.length; // 5
9         int arr2Len = arr2.length; // 4
10
11         int[] concatenatedArr = new int[arr1Len +
12
13         System.arraycopy(arr1, 0, concatenatedArr,
14
15         System.out.println(Arrays.toString(concatenatedArr));
16         // Concatenated array after array1 elements
17
18         System.arraycopy(arr2, 0, concatenatedArr,
19
20         System.out.println(Arrays.toString(concatenatedArr));
21     }
22 }
```

Collapse ^

Output

```
[12, 4, 5, 2, 5, 0, 0, 0, 0]
[12, 4, 5, 2, 5, 6, 10, 11, 6]
```

In the above code,

- `concatenatedArr` has memory for storing 9 integer values.

- `arraycopy(arr1, 0, concatenatedArr, 0, 5)` method tells the program to copy **5** elements of `arr1` starting from index **0** to `concatenatedArr` starting from index **0**.
- The first output of the code shows the
`concatenatedArr` after `arr1` elements are copied to it.
- `arraycopy(arr2, 0, concatenatedArr, 5, 4)` method tells the program to copy **4** elements of `array2` starting from index **0** to `concatenatedArr` starting from index **5**.
- The second output of the code shows the
`concatenatedArr` after `arr2` elements are copied to it.

▼ Comparison with Python

In Python, we can use

`+` operator to concatenate two lists.

Code

PYTHON

```
1 list1 = [12, 4, 5, 2, 5]
2 list2 = [6, 10, 11, 6]
3 concatenated_list = list1 + list2
4 print(concatenated_list)
```

Output

```
[12, 4, 5, 2, 5, 6, 10, 11, 6]
```

```
[
```

2. Array Slicing

Array slicing is a method of obtaining a subarray of an array.

We can get the subarray from an array using

`Arrays.copyOfRange()` method.

Syntax

JAVA

```
1 Arrays.copyOfRange(arrayName, startIndex, endIndex)
```

Here,

- `arrayName` is the name of the array
- The subarray starts from
`startIndex` and stops at `endIndex - 1`
- `endIndex` is not included in the subarray.

Code

JAVA

```
1 import java.util.Arrays;
2
```

```
3 class Main {  
4     public static void main(String[] args) {  
5         int[] originalArr = { 1, 2, 3, 4, 5 };  
6         int startIndex = 2;  
7         int endIndex = 4;  
8  
9         // copying the subarray from the origin  
10        int[] slicedArr = Arrays.copyOfRange(or
```

Expand ▼

Output

```
[3, 4]
```

▼ Comparison with Python

In Python, with the help of the colon

: operator, we can slice our list by specifying the start and end index.

Code

```
1 original_list = [1, 2, 3, 4, 5]  
2 sliced_list = original_list[2 : 4]  
3 print(sliced_list)
```

PYTHON

Output

```
[3, 4]
```

3. Multi-dimensional Array

A multi-dimensional array consists of an array of arrays. It is similar to the list of lists in Python.

We can create

N Dimensional arrays like 2D, 3D, etc.

In the previous unit, we learned one-dimensional arrays.

Now, let's look at the two-dimensional (2D) array.

3.1 Two-dimensional Array

A two-dimensional array is the simplest form of a multidimensional array. A two-dimensional array is an array of arrays. In other words, a two-dimensional array is a collection of one-dimensional arrays.

A two-dimensional array can be visualized as a matrix, which can be represented as a collection of rows and columns.

```
{  
  {12, 24, 36},  
  {15, 30, 45},  
  {16, 32, 48}  
}
```

Here,

- this two-dimensional array is a collection of **3** one-dimensional arrays
- `rows` = number of one-dimensional arrays = **3**
- `columns` = number of elements in each row = **3**
- number of elements in two-dimensional array = (number of rows * number of columns) = $3 * 3 = 9$
- As we know index in an array starts from **0**. So indexing of rows and columns also starts from **0**.
- The array

`{12, 24, 36}` is the 1st row of the two-dimensional array, for this complete row every element has its row index as **0** and the column index will lie from **0** to **2**.

- **12** is stored at the 0th row and 0th column of the array
- **24** is stored at the 0th row and 1st column of the array and so on for further elements for this row.
- The array

`{15, 30, 45}` is the 2nd row of a two-dimensional array, for this complete row every element has its row index as **1** and the column index will lie from **0** to **2**.

- **15** is stored at the 1st row and 0th column of the array
- **30** is stored at the 1st row and 1st column of the array and so on for further elements for this row.

3.1.1 Declaration of Two-dimensional Array

The syntax for the declaration of a two-dimensional array is as follows:

Syntax

JAVA

```
1 dataType[][] arrayName;
```

The syntax is similar to the one-dimensional array. The

`[][]` after the `dataType`, differentiates it from the one-dimensional array and regular variable declarations.

The number of square braces (

`[]`) indicates the dimensions. For example, three-dimensional (3D) array is declared as `dataType[][][] arrayName`

Code

JAVA

```
1 class Main {
2     public static void main(String[] args) {
3         int[][] arr; // declaring a two-dimensional
4     }
5 }
```

The

`arr` is a two-dimensional array that can store elements of type `int`.

4. Creating & Initializing a Two-dimensional Array

Using array literals we can declare, create and initialize a two-dimensional array in a single statement.

The following is the syntax to create a two-dimensional array using array literals:

Syntax

JAVA

```
1 dataType[][] arrayName = { {value00, value01, ...},
```

Here,

- value00 : is the value that will be stored at the 0th row and 0th column of the array arr .
- value01 : is the value that will be stored at the 0th row and 1st column of the array arr and so on.

Example:

Code

JAVA

```
1 class Main {  
2     public static void main(String[] args) {  
3         int[][] arr = {  
4             {45, 42},  
5             {11, 34},  
6             {56, 79}  
7         };  
8     }  
9 }
```

In the above code,

- The number of rows is **3**
- The number of columns is **2**
- 0th row and 0th column of array arr contain the value **45**.
- 0th row and 1st column of array arr contain the value **42**.
- 1st row and 0th column of array arr contain the value **11** and so on.

▼ Comparison with Python

Code

PYTHON

```
1 data = [[45, 42], [11, 34] , [56, 79] ]
```

5. Accessing Two-dimensional Array Elements

We use row and column indices to access a particular element from a two-dimensional array.

Syntax

JAVA

```
1 arrayName[rowNo][colNo];
```

Code

JAVA

```
1 class Main {  
2     public static void main(String[] args) {
```

```
3         int[][] arr = {{12, 4, 5}, {16, 18, 20}};
4
5         System.out.println(arr[0][0]);
6         System.out.println(arr[1][1]);
7         System.out.println(arr[1][2]);
8     }
9 }
```

Output

```
12
18
20
```

▼ Comparison with Python

Code

PYTHON

```
1 data = [[12, 4, 5], [16, 18, 20]]
2 print(data[0][0])
3 print(data[1][1])
4 print(data[1][2])
```

Output

```
12
18
20
```

6. Printing a Two-dimensional Array

To print a two-dimensional array or any multi-dimensional array, we can use

`Arrays.deepToString()` method.

Code

JAVA

```
1 import java.util.Arrays;
2
3 class Main {
4     public static void main(String[] args) {
5         int[][] arr = {
6             {12, 4, 5},
7             {16, 18, 20}
8         };
9
10        System.out.println(Arrays.deepToString(arr));
11    }
12 }
```

Collapse ^

Output

```
[[12, 4, 5], [16, 18, 20]]
```

▼ Comparison with Python

Code

PYTHON

```
1 data = [ [12, 4, 5], [16, 18, 20] ]
2 print(data)
```

Output

```
[[12, 4, 5], [16, 18, 20]]
```

7. Creating a Two-dimensional Array using `new` Keyword

We can also create the two-dimensional array using a

`new` keyword. Using this method, we can create a two-dimensional array of the required number of rows and columns and later assign the values.

The following is the syntax for creating an array using a

`new` keyword:

Syntax

JAVA

```
1 dataType[][] arrayName = new dataType[rows][columns]
```

Here,

- `arrayName` : name of the two-dimensional array.
- `rows` : specifies the number of rows.
- `columns` : specifies the number of columns.

Example:

Let's create a two-dimensional array with the name

`arr` :

Code

JAVA

```
1 class Main {
2     public static void main(String[] args) {
3         int[][] arr;    // declaring an array
4         arr = new int[2][2]; // allocating memory
5     }
6 }
```

In the above code, the statement

`arr = new int[2][2]` allocates memory for storing $2 \times 2 = 4$ `int` values.

After creating a memory for the two-dimensional array, a default value of `0` is assigned to each element as the array is of

`int` data type.

7.1 Assigning Values

Let's assign the values to the two-dimensional array created using the

`new` keyword.

Code

JAVA

```
1  import java.util.Arrays;
2
3  class Main {
4      public static void main(String[] args) {
5          int[][] arr;    // declaring an array
6          arr = new int[2][2]; // allocating memory
7          // assigning values to first row
8          arr[0][0] = 12;
9          arr[0][1] = 46;
10
11         // assigning values to second row
12         arr[1][0] = 61;
13         arr[1][1] = 90;
14
15         System.out.println(Arrays.deepToString(arr));
16     }
17 }
```

Collapse ^

Output

```
[[12, 46], [61, 90]]
```

8. Iterating Over a Two-dimensional Array

We can use nested loops to iterate over a two-dimensional array.

8.1 Using nested for Loop

We can use nested

`for` loops to iterate over a two-dimensional array.

Code

JAVA

```
1  class Main {
2      public static void main(String[] args) {
3          int[][] arr = {
4              {12, 45, 54},
5              {16, 18, 20},
6              {23, 78, 53}
7          };
8
9          int rows = arr.length;
10         int cols = arr[0].length;
11
12         for (int i = 0; i < rows; i++) {
13             for (int j = 0; j < cols; j++) {
```



```
14         System.out.println("arr[" + i + "]|
15     }
16 }
17 }
18 }
```

Collapse ^

Output

```
arr[0][0]: 12
arr[0][1]: 45
arr[0][2]: 54
arr[1][0]: 16
arr[1][1]: 18
arr[1][2]: 20
arr[2][0]: 23
arr[2][1]: 78
arr[2][2]: 53
```

In the above code,

- The array

`arr` contains 3 rows and each row contains 3 elements. Hence, rows = 3 and cols = 3.

- The outer for loop iterates from

`i = 0` to `i = 2`, this loop handles the row number.

- The inner for loop iterates from

`j = 0` to `j = 2`, this loop handles the column number.

- At

`i = 0`, and

- `j = 0`, `System.out.println(arr[0][0])` will print the value of 0th row and 0th column value of the array `arr`. Similarly, it runs till `j = 2`.
- `j = 3`, the inner loop test condition fails and control goes to the outer loop now.

Similarly, the corresponding row values will be printed for

`i = 1`, and `i = 2`

▼ Comparison with Python

Code

PYTHON

```
1 data = [[12, 45, 54], [16, 18, 20], [23, 78,
2 for i in range(len(data)):
3     for j in range(len(data[i])):
4         print(data[i][j])
```

Output

```
12
45
54
16
18
20
```

23

78

53

9. Read & Store User Inputs in a Two-dimensional Array

In the previous unit, we learned to read and store user inputs in a one-dimensional array. Similar to the one-dimensional array, initially, we need to create an array (two-dimensional array) to store the user inputs.

We have used

`for` loop to read user inputs in a one-dimensional array. For a two-dimensional array, we need to use nested `for` loops.

We need to get the number of rows and columns required to create the two-dimensional array. Let's consider that the first two numbers indicate the rows and columns, followed by the actual values to be stored.

Code

JAVA

```
1  import java.util.*;
2
3  class Main {
4      public static void main(String[] args) {
5          Scanner input = new Scanner(System.in);
6
7          int rows = input.nextInt(); // reads the
8          int cols = input.nextInt(); // reads the
9
10         int[][] arr = new int[rows][cols]; // cr
11
12         for (int i = 0; i < rows; i++) {
13             for (int j = 0; j < cols; j++) {
14                 arr[i][j] = input.nextInt(); //
15             }
16         }
17
18         System.out.println(Arrays.deepToString(arr
19         input.close();
20     }
21 }
```

Collapse ^

Input 1

```
2 2
16 64
32 128
```

Input 2

2
2
16
64
32
128

Output

[[16, 64], [32, 128]]

The above code works the same for **Input 1** and **Input 2**.

▼ Comparison with Python

Code

```
1 rows = int(input())
2 cols = int(input())
3 data = []
4 for i in range(rows):
5     list_a = []
6     for j in range(cols):
7         item = int(input())
8         list_a.append(item)
9     data.append(list_a)
10 print(data)
```

PYTHON

Input

2
2
16
64
32
128

Output

[[16, 64], [32, 128]]

Summary

-